

The .opencode Multi-Agent Harness

A staged, gated, self-correcting multi-agent loop that cuts hot-path instruction count — and proves every win.

research → plan-review (gate) → code → code-review (gate) → A/B-on-hardware → learn

What we'll walk through

1

The big picture

where the harness sits

2

Why a staged harness

the problem it solves

3

Three entry routes

the front door

4

The 7-stage pipeline

the spine + the gates

5

Hub-and-spoke + back-edges

how the loop self-corrects

6

Handoffs, skills, research

the connective tissue

7

A/B validation on hardware

proving the win

8

Memory + compaction safety

how it learns & survives

9

How to actually run it

presets & slash commands

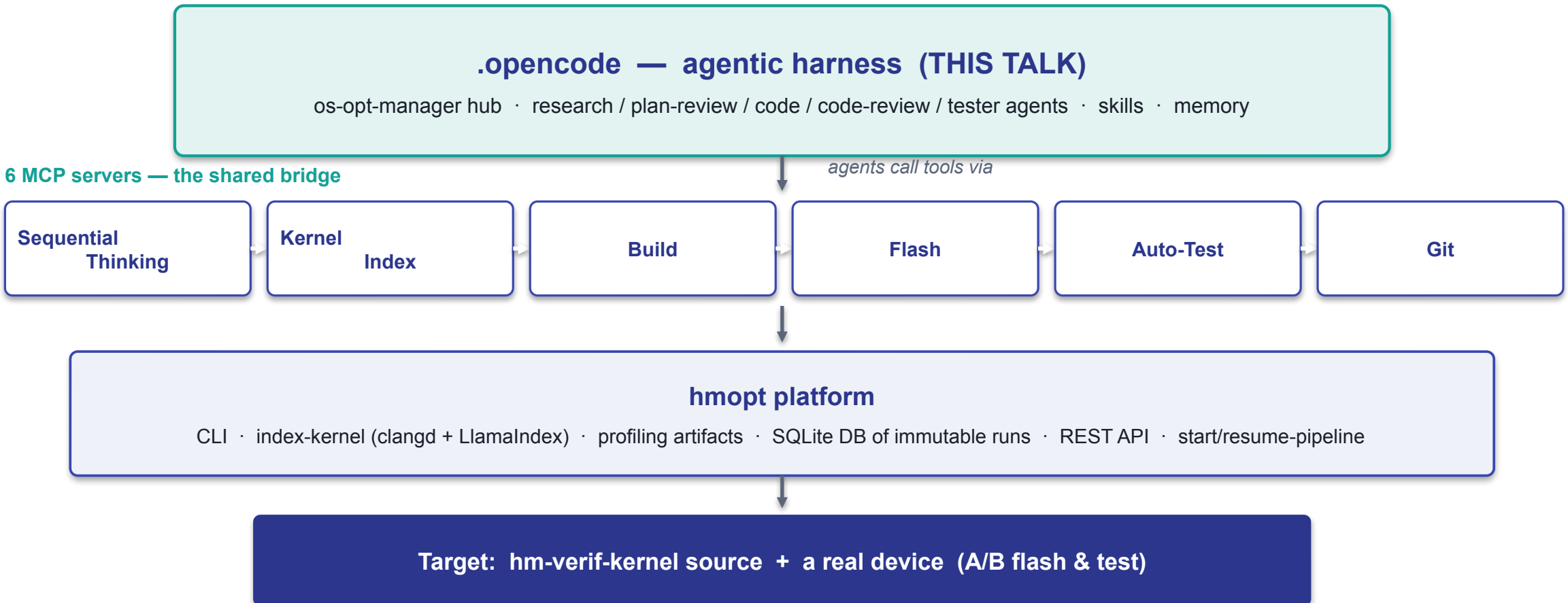
10

Artifacts & takeaways

what a run leaves behind

Where the harness sits

hmopt = the platform (the factory floor). **.opencode** = the agentic brain that drives it (this talk).



Why a staged harness — not one big agent

A single free-running agent optimizing a kernel fails in four predictable ways. Each gets a structural fix:

Agents drift out of lane

- A researcher starts coding; a coder grades its own work.
- Fix: one agent per stage, strict ownership

Work skips verification

- "Looks right" patches land with no measurement.
- Fix: mandatory plan + code gates, A/B on hardware

Long runs lose context

- Compaction wipes mid-task state; the agent forgets.
- Fix: state on disk — file wins over chat

"Wins" can't be trusted

- No falsifiable metric; vibes, not numbers.
- Fix: instruction-count delta, measured stock vs feature

Why instruction count (not latency)? It's deterministic, reproducible on noisy hardware, attributable per function, and falsifiable.

Three entry routes for three kinds of work

1 · Full automated pipeline

- @os-opt-manager / /optimize_*
- End-to-end: research → ... → decision
- Manager delegates every stage
- Use when: target is understood and you want a land-it run

2 · Human-in-the-loop

- @kernel-research (/research)
- @kernel-plan (/plan)
- Expert iterates turn-by-turn; living design doc + plan
- Use when: expert-driven exploration & planning

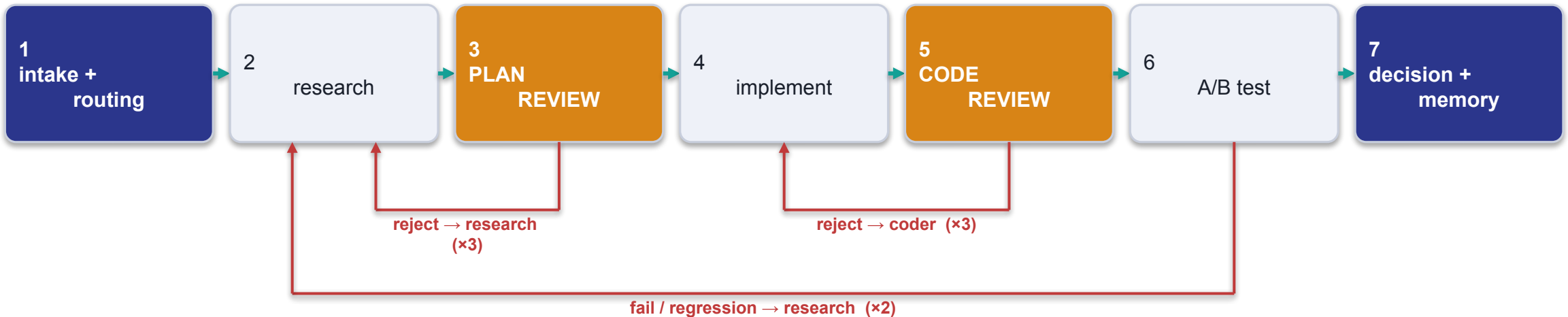
3 · Per-function deep dive

- @kernel-function-research (/function_detail)
- One-shot: design + multi-level callee graph
- Explain-only, single pass
- Use when: you need to understand ONE function fast

Routes 2 → 1 chain: @kernel-plan writes a plan that /optimize_generic picks up, implements, and tests.

The 7-stage gated pipeline

Green = the success path (runs in order). Red = exception branches — a failed gate or test loops back to one upstream owner.



■ mandatory gate

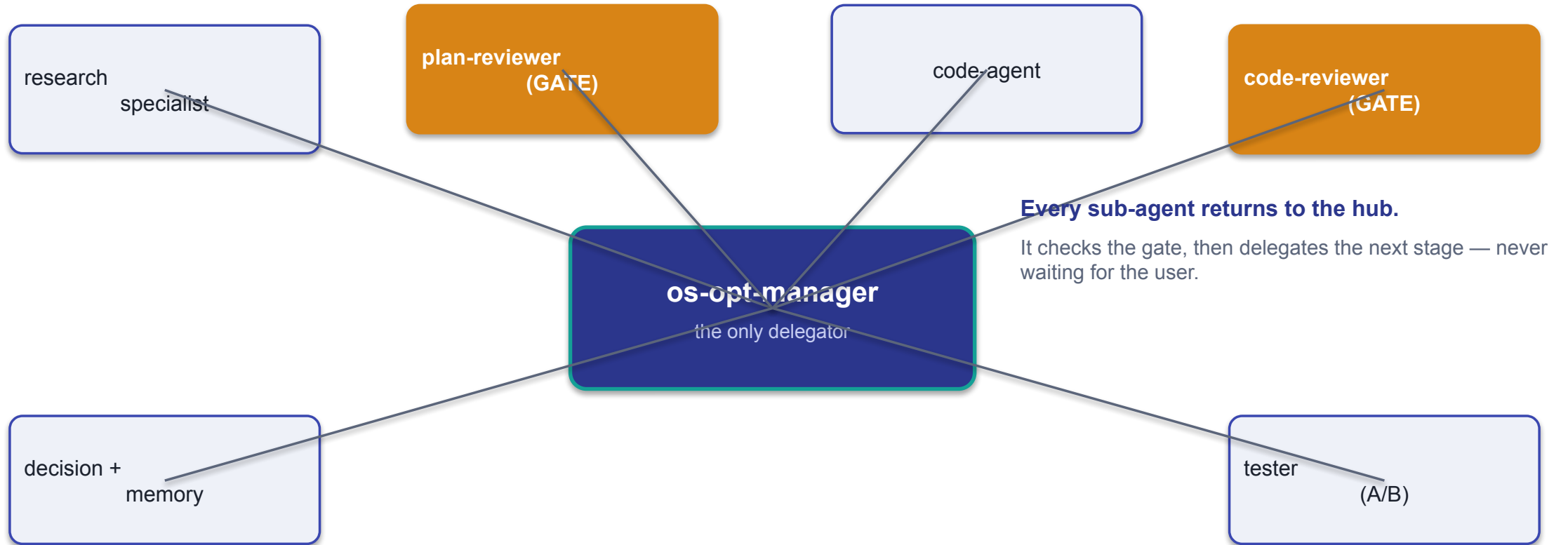
■ manager-owned stage

■ failure -> routes back to one upstream owner

On success vs. failure

- Forward only on success. A gate or test failure routes back to exactly one upstream owner — with loop caps (plan 3 · code 3 · test 2).
- Exceed a cap -> the run stops and surfaces to a human. A clean final PASS can auto-start the next iteration (Auto-Iterate).

Hub-and-spoke: the manager owns the loop



#1 failure mode it guards against: hallucinated delegation

- The manager must make a real `delegate()` tool call — not print a "Delegation to X" markdown block and stop.
- Verify it's real: sub-agent identity banner · status-line switch · new artifact on disk · `delegate()` in the tool trace.

Two mandatory gates + the A/B rule

Plan Review (gate)

- BEFORE any code is written
- approve / needs-revision / reject
- Bad-plan gate: rejects ideas already rejected or already landed
- Scope gate: rejects if the structural audit / scope justification is missing

Code Review (gate)

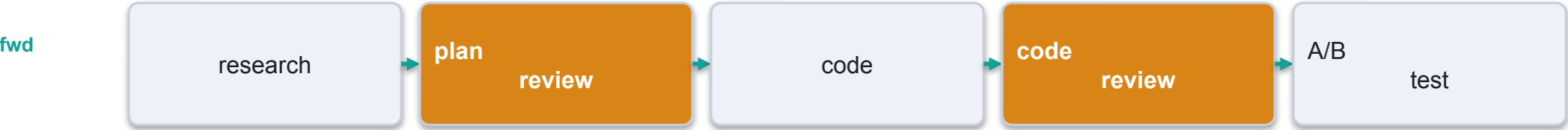
- AFTER every change, before acceptance
- approve / needs-revision / reject
- Checks added branches/loads/stores, deadlocks, leaks, lifecycle, scope
- Emits tester decision: required / recommended / skipped

A/B Test rule

- Flash AND test BOTH images: stock + feature
- A single-image result is FORBIDDEN as a verdict
- Verdict from delta: ≤ 0 = PASS · > 0 = FAIL · within $\pm 1\%$ = INCONCLUSIVE
- Per-modified-function compare, not just aggregate

The gates are the whole point: "no code without a reviewed plan; no win without a measurement."

Closed-loop: failures route back, wins iterate



Failing stage	Verdict	Routes back to	Cap
Plan review	needs-revision / reject	researcher (new mechanism)	3
Code review	needs-revision / reject	coder, or researcher if plan is flawed	3
A/B test	regression / target gone	researcher — thesis disproven	2
A/B test	within noise / too small	coder or researcher (per-pair table)	2

Auto-iterate: N

- A clean PASS auto-launches pass K+1 on the same target
- Prior wins treated as LANDED context
- Researcher must find orthogonal new mechanisms
- Slugs: base_slug, base_slug__iter2, ... (no clobber)
- Stops on: N reached · no_more_ideas · two passes in noise

Every bounce carries the prior artifacts + the failure reason + a loop counter. Hit the cap → stop and surface to the human.

The handoff packet contract

One shared packet on every handoff + each transition adds a few of its own fields.

Common packet — every handoff

- target + subsystem
- primary metric (instruction count)
- hot path + evidence source
- files / functions / structs in scope
- hypothesis + constraints (correctness · locks · lifetime)
- open questions
- next action + required reading

Each transition adds (key extra fields)

Transition	Adds, on top of the common packet
Research → Plan review	IC hypothesis · baseline evidence · rejected alternatives
Plan review → Coder	decision · must-keep semantics · must-not-cross
Coder → Code review	changed files · Modified functions list
Code review → Tester	stock + feature image paths · compare_level
Tester → Manager	verdict · delta_pct · recommended next route

The chat return stays tiny — verdict · artifact paths · a few file:line. The full design / plan / review / patch lives on disk.

Skill packs: shared rules, inlined once



Sub-agents must NOT re-Read skill files — the content is already in context (relative paths can resolve to the wrong/stale file).

Objective	Process	Gates
instruction-count-first	- research-discipline - optimization-funnel	- stage_gate_enforcement - handoff-contract - implementation-guardrails
Validation	Loop & memory	i18n
- build-and-sign - flash-device-operations - ab-test-comparison - validation-flight-check	- iterative-optimization - memory-accumulation - human-interaction-memory	language-config

~13 skills load on a full run. They're plain repo-local .md packs — vendor-neutral, version-controlled with the code.

Research & ideation discipline

Fixed research order (no shortcuts)

- Sequential-Thinking MCP → Kernel-Index MCP → local source
- → structural design doc → 5-dimension Structural Audit
- → instruction-count hypothesis → optimization

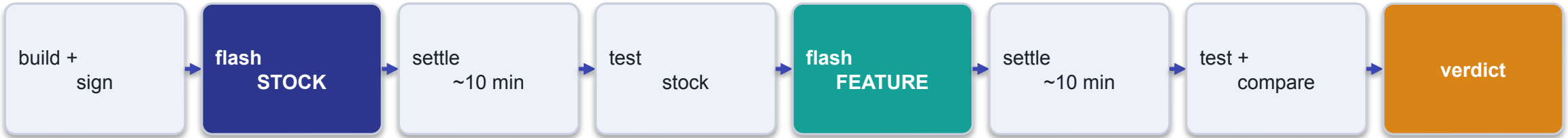
The 5-idea optimization funnel

- Generate exactly 5 ideas; ≥3 distinct scope tags required
- Dedup is a FILE check: bad_plans + memory + idea ledger
- Rank by hot-path IC win, then risk/cost; surface the top

Agent	Kind	Human-loop?	Ideates?	Writes plan?
kernel-research	primary	yes	no (explain only)	no
kernel-plan	primary	yes	yes (funnel)	yes
kernel-function-research	primary	no (1-shot)	no	no
kernel-source-research	pipeline subagent	no	yes	yes
domain specialists ×4	pipeline subagent	no	yes (ranked)	yes

Specialists (×4): memmgr-reclaim · hyperhold-io · sync-mechanism · workqueue-threadpool — auto-picked by keyword routing.

A/B validation on real hardware



A mandatory ~10-min settle follows every flash (stock and feature) before its test — never parallelize the two images.

Flash topology (real devices)

- Build server —pscp→ Windows relay (:9100) —USB→ phone
- Flash MCP (:7337) orchestrates; relay runs fastboot/hdc locally
- flash_pipeline.py = one deterministic local process (no HTTP hops)
- Mandatory preflight: relay health · device visible · creds

Test discipline

- Async only: run_*_async + 60 s polling (runs 30–120 min)
- 180-min ceiling; never declare a verdict while pending
- Build/sign fail = FAIL (broken patch)
- Relay/device infra fail = SKIPPED (not the patch's fault)

This is the part that touches the real world — and why a "win" here is trustworthy, not a guess.

Cross-run memory + the idea ledger

targets/ · subsystems/ · global_lessons

- Prose facts at 3 scopes (file → subsystem → repo)
- Stable structure, hot paths, good directions
- Loaded BEFORE fresh research; written at end of a run

idea_ledger/ (per target)

- Every verdicted mechanism, stable IDs (L001...)
- Statuses: approved / landed / rejected / deferred / reverted
- Landed rows carry delta_pct + compare level + validation path
- Feeds funnel dedup · never deleted

human_decisions/ · bad_plans

- Chronological log of every human review turn
- Purposes: audit · resume-after-compaction · dedup
- bad_plans = mechanisms not to re-propose

Two axes: by scope (target → subsystem → global) and by kind (prose facts · decision log · structured idea rows). Lifetime: forever.

Compaction-proof by design

Agentic runs are long and context gets compacted. The harness assumes it — and writes everything down.

The four moves

- File wins over chat — rebuild state from disk every turn
- `current_task.json` = the manager's source of truth
- Write to disk BEFORE acting (and before asking a human)
- Verbatim ANCHOR block each turn so recency survives compaction
- Resume a dead session straight from the state files

`.opencode/state/current_task.json`

```
"current_stage": "code_review",
"profile": "hyperhold_full",
"active_agent": "kernel-code-reviewer",
"gates_passed": ["plan_review:iter1"],
"pending_action": {
  "next_agent": "kernel-tester-agent",
  "expected_artifact": "..._validation.md"
},
"auto_iterate": { "current_iteration": 1,
  "iteration_history": [...] }
```

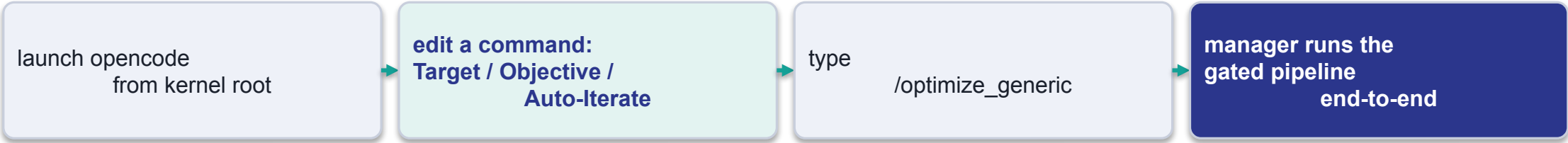
Same idea across the harness: compact handoffs, the idea ledger, and the decision log all exist so nothing is lost.

The MCP servers agents call

MCP server	Port	What agents use it for
Sequential Thinking	7333	Decompose & plan before acting (used first)
Kernel Index	7331	Symbol graph, call-chain, snippets, impact radius
Build	7335	kernel_build_trigger + kernel_sign_trigger (feature image)
Flash	7337	Flash stock/feature images via the Windows relay (:9100)
Auto-Test	7336	run_instruction_test_async + compare_reports_async
Git	7334	Diff / working-tree state for the coder & reviewers

The Kernel Index MCP is itself built by the hmopt platform (hmopt index-kernel: clangd + LlamaIndex) — the cleanest platform↔harness seam.

Running a pipeline: presets + slash commands



Preset / command	Target subsystem	Specialist
generic_full (default)	any directory / file (auto-routed)	kernel-source-research
hyperhold_full	swap / hyperhold I/O	hyperhold-io-opt
memmgr_reclaim_full	reclaim + allocator coupling	memmgr-reclaim-research
workqueue_full	workqueue / thread-pool loops	wq-threadpool-opt
sync_review (review-only)	lock scope / refcount / races	basic-mechanism-sync-opt

Also

- /research, /plan, /function_detail = understanding-only
- research_only = stop before code
- Language: zh-CN / en in config.yaml (prose only, not code)

A command @-inlines the preset + ~13 skills + memory + config into one prompt — that's how the whole pipeline boots from one keystroke.

What a run leaves on disk

.opencode/docs/

<target>_design.md — structure + Structural Audit

.opencode/plans/

<target>_plan.md — approved mechanisms only

.opencode/reviews/

_plan_review.md + _code_review.md (the gate verdicts)

.opencode/patches/

<topic>.patch + the actual source edits

.opencode/bench/

_validation.md (A/B delta) · iteration_summary · stall reports

.opencode/memory/

targets · subsystems · global_lessons · human_decisions · idea_ledger

.opencode/state/

current_task.json · iteration handoffs (the resume point)

Every gate, decision, and result is a file — so a run is fully auditable and replayable, not a black box.

TAKEAWAYS

- A staged, gated pipeline makes an LLM optimize a kernel the way a disciplined team does.
- Two hard gates + A/B-on-hardware: no code without a reviewed plan, no win without a measurement.
- Hub-and-spoke + back-edges: one manager drives the loop and self-corrects within caps.
- Engineered for long runs: state on disk, compact handoffs, resume-after-compaction.
- It learns: cross-run memory + idea ledger dedup every future proposal.
- One keystroke to run: a slash command inlines the preset, skills, memory, and config.